

Инструкция по настройке Postman

Что такое Postman и зачем он нужен

Postman - это инструмент для тестирования API. Он позволяет отправлять HTTP-запросы (GET, POST, PUT, DELETE) к вашему серверу и просматривать ответы в удобном формате, без необходимости писать код.

Аналогия: Если браузер - это инструмент для просмотра веб-сайтов, то Postman - это инструмент для общения с API. Браузер умеет только GET-запросы, а Postman - все методы, с любыми заголовками, телом и параметрами.

Шаг 1. Скачивание и установка Postman

1.1. Скачайте установщик

Откройте официальный сайт: <https://www.postman.com/downloads/>

Выберите версию для вашей ОС:

Windows - .exe установщик

macOS - .dmg или .zip (Apple Silicon / Intel)

Linux - .tar.gz или через snap

Скачайте файл.

1.2. Установите Postman

Windows:

Запустите скачанный .exe файл

Следуйте инструкциям установщика

После установки Postman запустится автоматически

macOS:

Откройте .dmg файл

Перетащите Postman в папку Applications

Запустите из Launchpad

Linux:

bash

Через snap

sudo snap install postman

Или распакуйте .tar.gz

`tar -xzf Postman-linux-x64.tar.gz`

`./Postman/Postman`

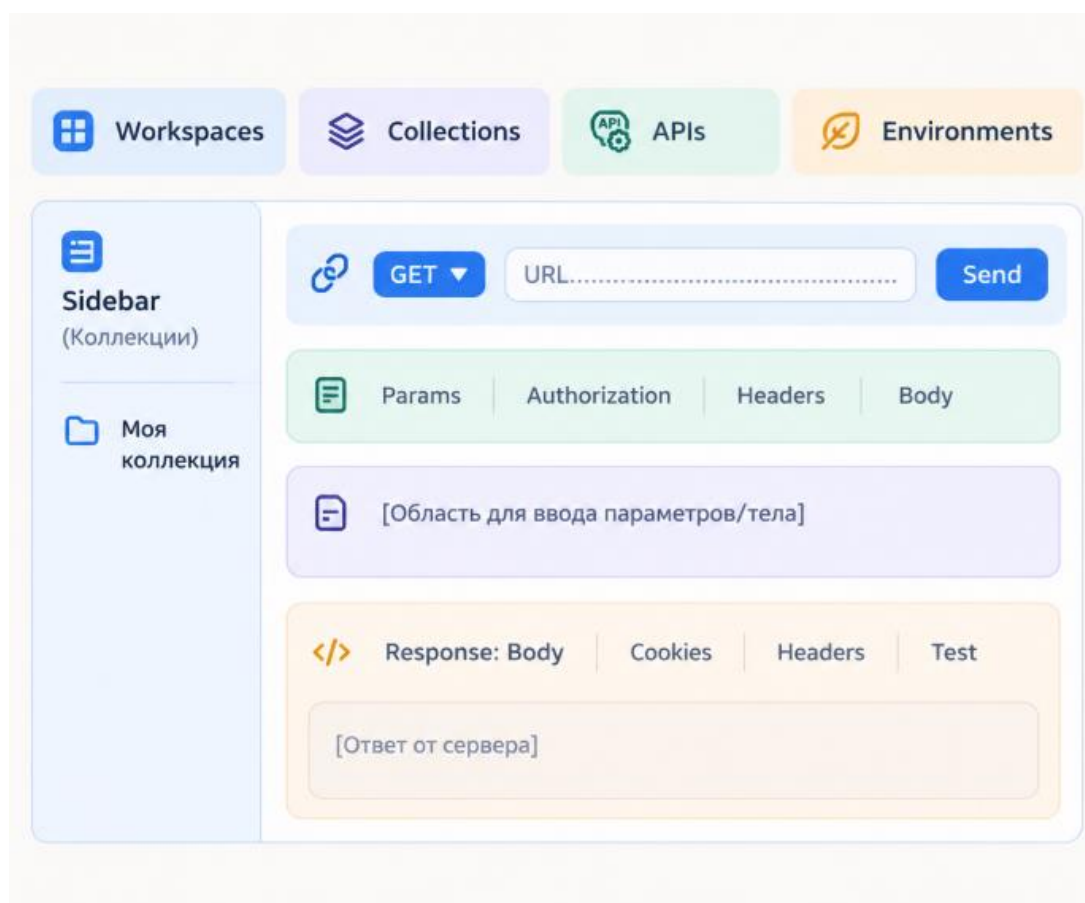
1.3. Первый запуск

Postman предложит войти или создать аккаунт - можно пропустить, нажав "Skip and take me to the app" (работает и без аккаунта).

Откроется главное окно Postman.

Шаг 2. Знакомство с интерфейсом

Основные элементы интерфейса



Элемент	Назначение
Sidebar (слева)	Список коллекций и запросов

Элемент	Назначение
Method (GET)	Выбор HTTP-метода
URL	Адрес запроса
Send	Кнопка отправки запроса
Params	Query-параметры (?key=value)
Authorization	Настройки аутентификации
Headers	HTTP-заголовки
Body	Тело запроса (для POST/PUT)
Response	Область ответа от сервера

Шаг 3. Создание первой коллекции

Коллекция - это папка для группировки связанных запросов. Удобно хранить все запросы к одному API в одной коллекции.

3.1. Создайте коллекцию

В левой панели нажмите "Collections" (или иконку папки)

Нажмите кнопку "+" или "New" → "Collection"

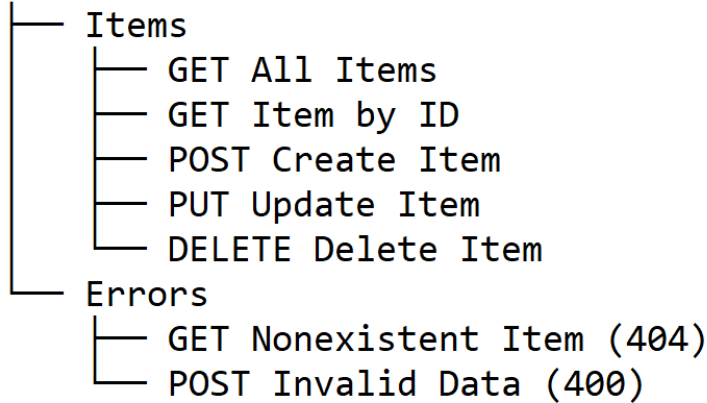
Введите название: "Lab2 CRUD API"

Нажмите "Create"

3.2. Структура коллекции для ЛР №2

Рекомендую создать такую структуру:

Lab2 CRUD API



Как создать папку внутри коллекции:

1. Нажмите правой кнопкой на коллекцию
2. Выберите "Add folder"
3. Введите название (например, "Items")

Шаг 4. Создание GET-запроса

4.1. Создайте новый запрос

Нажмите "New" → "HTTP Request" (или Ctrl+N)

Или нажмите правой кнопкой на папку Items → "Add request"

4.2. Настройте GET-запрос

Метод: выберите GET из выпадающего списка

URL: введите `http://localhost:3000/items`

Нажмите "Send"

4.3. Просмотрите ответ

В нижней части окна появится ответ:

json

```
{
  "count": 3,
  "items": [
    { "id": 1, "name": "Товар 1", "price": 100, "quantity": 5 },
    { "id": 2, "name": "Товар 2", "price": 200, "quantity": 3 },
    { "id": 3, "name": "Товар 3", "price": 300, "quantity": 10 }
  ]
}
```

Что можно увидеть в ответе:

Вкладка	Что показывает
Body	Тело ответа (JSON, HTML, текст)
Cookies	Cookies, установленные сервером
Headers	Заголовки ответа
Test Results	Результаты автотестов (если настроены)
Status	Код ответа (200 OK)
Time	Время выполнения запроса
Size	Размер ответа

4.4. Сохраните запрос

Нажмите "Save" (или Ctrl+S)

Введите название: "GET All Items"

Выберите коллекцию "Lab2 CRUD API" и папку "Items"

Нажмите "Save"

Шаг 5. Создание GET-запроса с параметром

5.1. Создайте новый запрос

Метод: GET

URL: http://localhost:3000/items/1

5.2. Отправьте запрос

Нажмите "Send". Ответ:

json

```
{ "id": 1, "name": "Товар 1", "price": 100, "quantity": 5 }
```

5.3. Проверьте обработку 404

Измените URL на `http://localhost:3000/items/999` и нажмите "Send".

Ответ (404 Not Found):

json

```
{ "error": "Элемент не найден" }
```

Обратите внимание на код ответа в правом верхнем углу - он будет 404 Not Found (красный).

Шаг 6. Создание POST-запроса (создание элемента)

6.1. Создайте новый запрос

Метод: POST

URL: `http://localhost:3000/items`

6.2. Настройте тело запроса

Перейдите на вкладку "Body" (под URL)

Выберите "raw"

В выпадающем списке справа выберите "JSON" (важно!)

Введите JSON:

json

```
{  
  "name": "Новый товар из Postman",  
  "price": 500,  
  "quantity": 7  
}
```

6.3. Отправьте запрос

Нажмите "Send".

Ответ (201 Created):

json

```
{  
  "id": 4,  
  "name": "Новый товар из Postman",
```

```
"price": 500,  
"quantity": 7  
}
```

6.4. Проверьте, что элемент создан

Создайте новый GET-запрос на <http://localhost:3000/items> - в списке должен появиться новый элемент.

Шаг 7. Создание PUT-запроса (обновление элемента)

7.1. Создайте новый запрос

Метод: PUT

URL: <http://localhost:3000/items/1>

7.2. Настройте тело запроса

Вкладка "Body" → "raw" → "JSON"

Введите:

```
json  
  
{  
  "name": "Обновлённый товар",  
  "price": 150,  
  "quantity": 20  
}
```

7.3. Отправьте запрос

Ответ (200 OK):

```
json  
  
{  
  "id": 1,  
  "name": "Обновлённый товар",  
  "price": 150,  
  "quantity": 20  
}
```

Шаг 8. Создание DELETE-запроса (удаление элемента)

8.1. Создайте новый запрос

Метод: DELETE

URL: `http://localhost:3000/items/3`

8.2. Отправьте запрос

Ответ (200 OK):

json

```
{  
  "message": "Элемент удалён",  
  "deleted": { "id": 3, "name": "Товар 3", "price": 300, "quantity": 10 }  
}
```

8.3. Проверьте, что элемент удалён

GET-запрос на `http://localhost:3000/items` - элемента с `id=3` больше нет.

Шаг 9. Использование переменных окружения (Environments)

Переменные окружения позволяют не писать URL каждый раз, а использовать переменную `{{base_url}}`.

9.1. Создайте окружение

В левой панели нажмите "Environments"

Нажмите "+" или "Create Environment"

Назовите: "Local Development"

Добавьте переменные:

Variable	Initial Value	Current Value
<code>base_url</code>	<code>http://localhost:3000</code>	<code>http://localhost:3000</code>

Нажмите "Save"

9.2. Активируйте окружение

В правом верхнем углу выберите "Local Development" из выпадающего списка.

9.3. Используйте переменную в запросах

Вместо `http://localhost:3000/items` пишите:

```
{{base_url}}/items
```

Postman автоматически подставит значение из окружения.

Шаг 10. Автоматические тесты в Postman

10.1. Что такое тесты

Тесты - это JavaScript-код, который автоматически проверяет ответ сервера.

10.2. Напишите первый тест

Откройте запрос "GET All Items"

Перейдите на вкладку "Tests" (рядом с Body)

Введите код:

```
javascript
```

```
// Проверка статуса ответа
```

```
pm.test("Статус 200 OK", function () {  
    pm.response.to.have.status(200);  
});
```

```
// Проверка, что ответ - JSON
```

```
pm.test("Ответ в формате JSON", function () {  
    pm.response.to.be.json;  
});
```

```
// Проверка наличия поля count
```

```
pm.test("Есть поле count", function () {  
    const jsonData = pm.response.json();  
    pm.expect(jsonData).to.have.property('count');  
});
```

// Проверка, что items - массив

```
pm test("items - это массив", function () {  
    const jsonData = pm response json();  
    pm expect(jsonData.items) to be an('array');  
});
```

10.3. Запустите запрос

Нажмите "Send". Перейдите на вкладку "Test Results" - все тесты должны быть PASS (зелёные).

10.4. Тесты для других запросов

POST Create Item (проверка 201):

javascript

```
pm test("Статус 201 Created", function () {  
    pm response to have status(201);  
});
```

```
pm test("Ответ содержит ID", function () {  
    const jsonData = pm response json();  
    pm expect(jsonData).to have property('id');  
    pm expect(jsonData.id).to be a('number');  
});
```

GET Nonexistent Item (проверка 404):

javascript

```
pm test("Статус 404 Not Found", function () {  
    pm response to have status(404);  
});
```

```
pm test("Ответ содержит error", function () {  
    const jsonData = pm response json();  
    pm expect(jsonData).to have property('error');  
});
```

Шаг 11. Запуск всех тестов коллекции (Collection Runner)

11.1. Запустите Collection Runner

Нажмите правой кнопкой на коллекцию "Lab2 CRUD API"

Выберите "Run collection"

Откроется окно Collection Runner

11.2. Настройте запуск

Выберите все запросы (или отдельные)

Нажмите "Run Lab2 CRUD API"

11.3. Посмотрите результаты

Postman выполнит все запросы по порядку и покажет:

Запрос	Статус	Время
GET All Items	 PASS	45 ms
GET Item by ID	 PASS	32 ms
POST Create Item	 PASS	51 ms
PUT Update Item	 PASS	48 ms
DELETE Delete Item	 PASS	40 ms
GET Nonexistent Item (404)	 PASS	29 ms
POST Invalid Data (400)	 PASS	35 ms

Шаг 12. Экспорт и импорт коллекции

12.1. Экспорт

Нажмите правой кнопкой на коллекцию

Выберите "Export"

Выберите формат "Collection v2.1"

Сохраните файл Lab2_CRUD_API.postman_collection.json

Зачем: Чтобы сдать работу преподавателю или загрузить на другой компьютер.

12.2. Импорт

Нажмите "Import" (в левой панели)

Выберите файл .json

Коллекция появится в вашем Postman

Шаг 13. Полезные советы

13.1. Горячие клавиши

Клавиша	Действие
Ctrl + N	Новый запрос
Ctrl + S	Сохранить запрос
Ctrl + Enter	Отправить запрос
Ctrl + /	Комментарий в тестах

13.2. Просмотр заголовков

На вкладке "Headers" в ответе можно увидеть:

text

Content-Type: application/json; charset=utf-8

Content-Length: 145

Date: Mon, 15 Jan 2024 10:00:00 GMT

13.3. Сохранение примеров ответов

В ответе нажмите "Save Response" → "Save as example"

Пример сохранится вместе с запросом

Это полезно для документации

13.4. Просмотр истории запросов

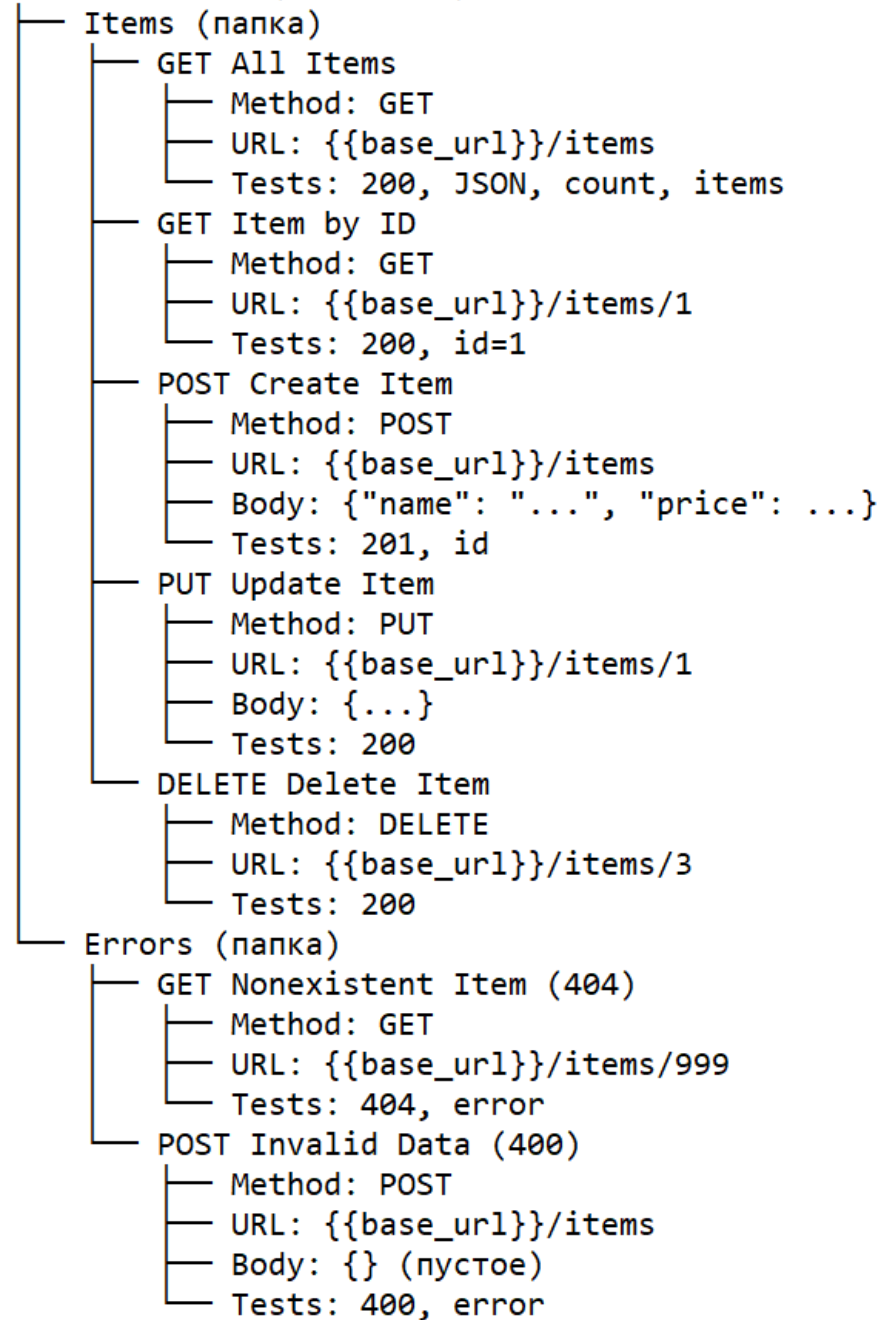
В левой панели нажмите "History" - увидите все выполненные запросы.

Шаг 14. Типичные ошибки и их решение

Ошибка	Причина	Решение
Could not get response	Сервер не запущен	Запустите <code>npm run dev</code>
ECONNREFUSED	Неверный порт	Проверьте, что сервер на <code>localhost:3000</code>
404 Not Found	Неверный URL	Проверьте маршрут в <code>app.js</code>
400 Bad Request	Неверное тело запроса	Проверьте JSON на вкладке Body
Unexpected token	Неверный JSON	Проверьте синтаксис JSON
Cannot read property	Обращение к несуществующему полю	Проверьте структуру ответа

Шаг 15. Итоговая структура коллекции для ЛР №2

Lab2 CRUD API (коллекция)



Что сдать в отчёте

1. Скриншот всех запросов из Postman с ответами.
2. Скриншот Test Results (все тесты PASS).
3. Экспортированную коллекцию (.json файл) - приложить к отчёту.
4. Скриншот Collection Runner с результатами всех тестов.

Рекомендуемые источники

1. Postman Learning Center - <https://learning.postman.com/>

2. Postman Documentation - <https://documenter.getpostman.com/>
3. Тестирование API в Postman (видео) - YouTube-каналы по тестированию ПО
4. REST API Testing with Postman - <https://www.postman.com/api-platform/api-testing/>
5. Postman - это инструмент, который вы будете использовать на протяжении всей карьеры бэкенд-разработчика.